

Functions

COSC 1210 (CS I) Fall 26 | Augustana U.

Table of Contents

- [1. Objectives](#)
- [2. Why functions?](#)
- [3. Anatomy of a function](#)
- [4. Parameters and arguments](#)
- [5. Documenting a function](#)
- [6. PRACTICE design a docstring](#)
- [7. Calling one function from another](#)
- [8. Generalizing a function](#)
- [9. Stacking functions](#)
- [10. PRACTICE receipt](#)
- [11. Repetition: for and range\(\)](#)
- [12. A loop inside a function](#)
- [13. PRACTICE rectangle and triangle](#)
- [14. Local variables live inside the function](#)
- [15. Reading a traceback](#)
- [16. PRACTICE debug the report](#)
- [17. Summary](#)
- [18. Glossary](#)

1. Objectives

Source: Downey, Think Python (3e), chapter 3.

By the end of this lesson you can:

- define your own functions with `def`
- tell a **parameter** from an **argument**, and give a parameter a default
- document a function with a **docstring** and read it back with `help`
- call one function from inside another
- repeat work with a `for` loop and `range()`
- explain why a variable made inside a function is invisible outside it
- read a traceback from the bottom up

Three sessions:

1. Defining, documenting and calling functions
2. Repetition: `for` and `range()`
3. Local variables and tracebacks

Chapter 3 functions only **print** - they do not hand a value back yet. Giving a value back (`return`) is chapter 6.

2. Why functions?

- A function **names** a group of statements, so the program reads more like a description of what it does.

- The code appears **once**. Fix a bug or change the wording in one place, and every call gets it.
- You can build and test the pieces **separately**, then assemble them.
- A good function is **reusable** in the next program.

3. Anatomy of a function

A function groups statements under a name so you can run them again by calling the name. The smallest useful one takes no input:

```
def welcome():
    print("Welcome to COSC 1210.")
    print("Today's topic: functions.")

welcome()
```

```
Welcome to COSC 1210.
Today's topic: functions.
```

- def is the keyword that starts a **definition**.
- welcome is the name - any legal variable name works.
- The empty () mean "no input".
- The header ends with a colon ; the **body** is indented (four spaces).
- Nothing happens until you **call** it: welcome().

Defining a function creates a value, like a number or a string - its type is function:

```
print(type(welcome))
```

```
<class 'function'>
```

4. Parameters and arguments

One parameter

To pass input in, put a **parameter** name in the parentheses:

```
def greet(name):
    print("Hello, " + name + "!")

greet("Jim")
greet("Joe")
```

```
Hello, Jim!
Hello, Joe!
```

- name is a **parameter** - it belongs to the definition.
- "Jim" is an **argument** - it belongs to the call.
- Calling greet("Jim") is the same as doing name = "Jim" and then running the body:

```
name = "Jim"
print("Hello, " + name + "!")
```

```
Hello, Jim!
```

An argument can be a variable, not just a literal:

```
student = "Joe"
greet(student)
```

```
Hello, Joe!
```

Two parameters

Two parameters, separated by commas (this time with an f-string):

```
def greet_course(name, course):
    print(f"Hello, {name}, welcome to {course}.")

greet_course("Jim", "COSC 1210")
```

```
Hello, Jim, welcome to COSC 1210.
```

Every definition has the same parts:

Part	In <code>greet_course</code>
keyword	<code>def</code>
name	<code>greet_course</code>
parameters	<code>name, course</code>
colon	<code>:</code>
body	the indented <code>print(...)</code>
call	<code>greet_course("Jim", "COSC 1210")</code>
arguments	<code>"Jim", "COSC 1210"</code>

Parameters live in the definition; **arguments** live in the call.

Default parameters

You've seen this in the Python documentation for built-in functions: some parameters are "positional" (Python expects them only there), others are "named":

Example:

```
help(round)
```

Help on built-in function round in module builtins:

```
round(number, ndigits=None)
    Round a number to a given precision in decimal digits.

    The return value is an integer if ndigits is omitted or None. Otherwise
    the return value has the same type as the number. ndigits may be negative.
```

The number parameter is **positional**, so that when you type `round(2.55)` it is recognized: `number = 2.55`.

The `ndigits` parameter for the number of digits to be kept after the decimal point is **named**.

We could also create a greeting function with a named parameter. Then it could be called either with or without an argument:

```
def hello(name="there"):
    print(f"Hello, {name}!")

hello()
hello("Jane")
```

```
Hello, there!
Hello, Jane!
```

5. Documenting a function

A string on the first line of the body is a **docstring** - a note about what the function does. `help()` shows it:

```
def greet(name):
    "Print a friendly hello for the given name."
    print("Hello, " + name + "!")

help(greet)
```

```
Help on function greet in module __main__:

greet(name)
    Print a friendly hello for the given name.
```

When a function has several arguments, arguments whose meaning is not obvious, or output that needs explaining, put that in the docstring too

- not just a one-line summary.

Compare `help(round)` again, this time the last two lines: they say what the function **gives back** and how its type depends on the input.

```
help(round)
```

Help on built-in function round in module builtins:

```
round(number, ndigits=None)
```

Round a number to a given precision in decimal digits.

The return value is an integer if ndigits is omitted or None. Otherwise the return value has the same type as the number. ndigits may be negative.

Here is the same idea for a user-defined function. `fahrenheit` takes a temperature and prints it in another unit - and `celsius` alone does not tell you whether the **input** or the **output** is in Celsius, so the docstring has to.

```
def fahrenheit(celsius):  
    """Print a Celsius temperature converted to Fahrenheit.  
  
    celsius -- a temperature in degrees Celsius (a number)  
    Prints the same temperature in degrees Fahrenheit; returns nothing.  
    """  
    print(celsius * 9 / 5 + 32)  
  
fahrenheit(100)  
help(fahrenheit)
```

```
212.0  
Help on function fahrenheit in module __main__:  
  
fahrenheit(celsius)  
    Print a Celsius temperature converted to Fahrenheit.  
  
    celsius -- a temperature in degrees Celsius (a number)  
    Prints the same temperature in degrees Fahrenheit; returns nothing.
```

The docstring has three parts, like the ones from `help(round)`:

- a one-line **summary** of what the function does;
- one line per **parameter**, saying what it stands for and its type;
- a line about the **output** - here, that it prints and hands nothing back.

6. **PRACTICE** design a docstring

Add a docstring to each function below that names its arguments and says what it does with them, then test both with `help`:

```
def hypotenuse(a, b):  
    c = (a**2 + b**2) ** 0.5  
    print(c)  
  
def discount(price, percent):  
    print(price - price * percent / 100)  
  
hypotenuse(3, 4)  
discount(50, 20)
```

```
5.0  
40.0
```

For discount, the docstring has to settle one real question: is percent the number 20 or the fraction 0.20?

Solution skip

```
def hypotenuse(a, b):
    """Print the hypotenuse of a right triangle.

    a, b -- the lengths of the two legs (numbers)
    Prints the hypotenuse; returns nothing.
    """
    c = (a**2 + b**2) ** 0.5
    print(c)

def discount(price, percent):
    """Print the sale price after taking a percentage off.

    price -- the original price (a number)
    percent -- the discount as a whole number, so 20 means 20% off
    Prints the reduced price; returns nothing.
    """
    print(price - price * percent / 100)

hypotenuse(3, 4)
help(hypotenuse)
discount(50, 20)
help(discount)
```

```
5.0
Help on function hypotenuse in module __main__:

hypotenuse(a, b)
    Print the hypotenuse of a right triangle.

    a, b -- the lengths of the two legs (numbers)
    Prints the hypotenuse; returns nothing.

40.0
Help on function discount in module __main__:

discount(price, percent)
    Print the sale price after taking a percentage off.

    price -- the original price (a number)
    percent -- the discount as a whole number, so 20 means 20% off
    Prints the reduced price; returns nothing.
```

7. Calling one function from another

Once a function exists, other functions can call it. Here notice calls rule twice and print once:

```
def rule():
    print("-" * 24)

def notice(text):
    rule()
    print(text)
    rule()
```

```
notice("Class cancelled Friday")
```

```
-----  
Class cancelled Friday  
-----
```

8. Generalizing a function

rule always prints 24 dashes, whatever the text underneath. We can make it fit the text by giving it a length parameter - keeping 24 as the default, so the plain rule() calls elsewhere still work:

```
def rule(length=24):  
    print("-" * length)  
  
def notice(text):  
    rule(len(text))  
    print(text)  
    rule(len(text))  
  
notice("Classes cancelled Friday and Monday!")
```

```
-----  
Classes cancelled Friday and Monday!  
-----
```

- rule now takes a length and prints that many dashes.
- notice works out the width it needs with len(text) and passes it in - once for each rule.
- Nothing in notice is tied to one message any more: give it a longer or shorter text and both rules follow.

This is **refactoring**: same behaviour, more general code.

9. Stacking functions

Functions can be stacked several deep. bulletin calls header, which calls rule, which calls print:

```
def header():  
    rule()  
    print("COSC 1210 - Bulletin")  
    rule()  
  
def bulletin(item):  
    header()  
    print("* " + item)  
    rule()  
  
bulletin("Homework due Monday")
```

```
-----  
COSC 1210 - Bulletin  
-----
```

```
* Homework due Monday
```

- The call chain is bulletin -> header -> rule -> print.
- You could write this with fewer functions; the point is that they fit together.

10. PRACTICE receipt

Write two functions:

1. `rule()` - prints a line of 24 dashes.
2. `receipt(item, price, qty)` - prints, in order: a rule; the line `<qty> x <item>`; the line `Total: $<price times qty>`; a rule. Use `rule()` for the rules and an f-string for the two middle lines.
3. Provide docstrings for both functions.

Then call `receipt("coffee", 3.5, 2)`.

Correct output:

```
-----
2 x coffee
Total: $7.0
-----
```

Solution [skip](#)

```
def rule():
    """Print a line of 24 dashes."""
    print("-" * 24)

def receipt(item, price, qty):
    """Print a receipt for one item.

    item -- a sold item (a string)
    price -- the item price (a number)
    qty -- the number of items sold (a number)
    Prints the <qty> x <item> line and the Total line (price * qty);
    returns nothing.
    Calls rule() for the top and bottom lines.
    """
    rule()
    print(f"{qty} x {item}\nTotal: ${price * qty}")
    rule()

receipt("coffee", 3.5, 2)

help(receipt)
```

```
-----
2 x coffee
Total: $7.0
-----
Help on function receipt in module __main__:

receipt(item, price, qty)
    Print a receipt for one item.
```



```
item -- a sold item (a string)
price -- the item price (a number)
qty -- the number of items sold (a number)
Prints the <qty> x <item> line and the Total line (price * qty);
returns nothing.
Calls rule() for the top and bottom lines.
```

11. Repetition: for and range()

To repeat statements, use a for loop.

```
for i in range(3):
    print(i)
```

```
0
1
2
```

- for is a new keyword; i is a new variable; in is a keyword.
- range(3) produces the numbers 0, 1, 2 - it **starts at 0** and **stops before 3**:

```
[print(i, end=' ') for i in range(10)];
```

```
0 1 2 3 4 5 6 7 8 9
```

- The header ends with : and the body is indented.
- Each time round: run the body with the next value of i, then go back. When range runs out, the loop ends.

The loop variable is usually part of the work:

```
for n in range(5):
    print(n, "squared is", n**2)
```

```
0 squared is 0
1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
```

But it does not have to be. Here the body ignores the counter - the loop just runs three times, and _ stands for a "dummy" variable:

```
for _ in range(3):
    print("Welcome!")
```

```
Welcome!
Welcome!
```

Welcome!

12. A loop inside a function

Put the loop in a function's body and it runs on whatever argument you pass:

```
def stars(n):  
    for i in range(n):  
        print("*" * (i + 1))  
  
stars(5)
```

```
*  
**  
***  
****  
*****
```

- The for body is indented twice: once for the function, once for the loop.
- `i + 1` because range starts at 0, so the first row gets one star.
- You should always trace for loops out - you can do that with a table like this, that shows loop counter and output:

i	output
0	*
1	**
2	***
3	****
4	*****

13. **PRACTICE** rectangle and triangle

1. Write `rectangle(char, width, height)` that prints a solid block: height rows, each row char repeated width times. Don't forget the docstring.

```
rectangle("#", 5, 3)  
#####  
#####  
#####
```

2. Write `triangle(char, height)` that prints a right triangle: row k has k copies of char, for k from 1 up to height. Don't forget the docstring.

```
triangle("#", 4)  
#  
##  
###  
####
```

Use def, a for loop, range, * and print. No if.

Solution skip

```
def rectangle(char, width, height):
    """Print a solid rectangle built from a repeated character.

    char -- the character to draw with, e.g. '#'
    width -- how many characters wide (a whole number)
    height -- how many rows tall (a whole number)
    Prints the block; returns nothing.
    """
    for row in range(height):
        print(char * width)

rectangle('#', 5, 3)
```

```
#####
#####
#####
```

```
def triangle(char, height):
    """Print a right triangle built from a repeated character.

    char -- the character to draw with, e.g. '#'
    height -- how many rows tall; row k has k characters (a whole number)
    Prints the triangle; returns nothing.
    """
    for row in range(height):
        print(char * (row + 1))

rectangle("#", 5, 3)
print()
triangle("#", 4)
```

```
#####
#####
#####

#
##
###
####
```

14. Local variables live inside the function

A variable created inside a function is **local**: it exists only while the function runs. The part of the program where the variable exists is also called its **scope**.

```
def make_total(price, qty):
    total = price * qty
    print("Total:", total)

make_total(3.5, 2)
```

```
Total: 7.0
```

Ask for total outside the function and it is gone because it belonged to make_total:

```
print(total)    # total belonged to make_total, which has finished
```

```
NameError: name 'total' is not defined
```

- Parameters (price, qty) are local too.
- This is a feature: two functions can use the same name without colliding.

```
def show_square(n):  
    answer = n * n  
    print(n, "squared is", answer)  
  
def show_cube(n):  
    answer = n * n * n    # a different 'answer', local to show_cube  
    print(n, "cubed is", answer)  
  
show_square(3)  
show_cube(3)
```

```
3 squared is 9  
3 cubed is 27
```

15. Reading a traceback

When an error happens **inside** a function, Python prints the whole chain of calls that led there. That is a **traceback**.

```
def make_tag(name):  
    print("Hello, " + nme + "!")    # typo: nme  
  
def welcome(name):  
    print("Welcome!")  
    make_tag(name)  
  
welcome("Ada")
```

```
Welcome!
```

Run as a script, this prints:

```
Welcome!  
Traceback (most recent call last):  
  File "functions.py", line 8, in <module>  
    welcome("Ada")  
  File "functions.py", line 6, in welcome  
    make_tag(name)  
  File "functions.py", line 2, in make_tag
```

```
print("Hello, " + nme + "!") # typo: nme
NameError: name 'nme' is not defined. Did you mean: 'name'?
```

Read it bottom to top:

```
NameError: name 'nme' is not defined    <- what went wrong
make_tag(...)                          <- where it happened
welcome(...)                            <- who called it
<module>                                <- the top-level script
```

- The actual error is always the last line - start there.
- Each line above is one step back up the chain of calls (the "stack").
- Python often suggests the fix: *Did you mean: 'name'?*

To see **why** a value is wrong, paste the code into pythontutor.com and step through it: welcome and make_tag appear as boxes stacked on top of the top-level code, each box showing the value of every variable at that moment, and you watch the error strike the top box - the traceback made visible. This is the tool to reach for when the numbers do not add up.

(In a notebook, the magic command `%xmode - Minimal, Plain, or the default Context` - trims or restores how much of the traceback prints; it changes the display only, not the error.)

(You will sometimes see `if __name__ == "__main__":` at the foot of a file. That is a topic for later, when programs span several files.)

16. **PRACTICE** debug the report

The program below prints a short class report. It has **three** bugs: one stops it before it runs, one crashes it partway, one is a scope mistake.

```
def divider():
    print("=" * 20)

def line(label, value)
    print(label + ": " + str(value))

def report(present, absent):
    divider()
    line("Present", present)
    line("Absent", absent)
    total = present + absent
    divider()

report(18)
print("Total students:", total)
```

When it works:

```
=====
Present: 18
Absent: 2
=====
Total students: 20
```

Step through it in pythontutor.com, fix the first error it reports, run again, and repeat. For each bug, write a one-line comment saying what was wrong.

There is a bonus assignment "Debug Bill Splitter" available that asks for a similar thing.

Solution skip

Bugs:

1. `def line(label, value)` - missing the colon -> `SyntaxError`, caught before anything runs.
2. `report(18)` - only one argument for two parameters -> `TypeError: report() missing 1 required positional argument: 'absent'`. Call it `report(18, 2)`.
3. `total` is local to `report`, so `print("Total students:", total)` outside it -> `NameError`. There is no return yet, so the fix is to print the total **inside** `report`.

```
def divider():
    print("=" * 20)

def line(label, value):
    print(label + ": " + str(value))

def report(present, absent):
    divider()
    line("Present", present)
    line("Absent", absent)
    total = present + absent
    divider()
    print("Total students:", total)

report(18, 2)
```

```
=====
Present: 18
Absent: 2
=====
Total students: 20
```

17. Summary

- `def name(parameters):` starts a function; the body is indented.
- Defining a function makes a value of type `function`; nothing runs until you **call** it.
- A **parameter** is a name in the definition; an **argument** is a value in the call. Calling is like assigning the argument to the parameter, then running the body.
- A parameter can carry a **default value** (`def rule(length=24)`); a call may then leave that argument out.
- A **docstring** - a string as the first line of the body - documents a function: a one-line summary, one line per parameter, and a line about the output. `help(f)` prints it.
- Functions can call other functions; that is how small pieces build a program.
- **Refactoring** is rewriting working code to be cleaner or more general
 - swapping a hard-coded 24 for a `length` parameter, say - without changing what it does.
- A for loop repeats its body. `range(n)` gives 0, 1, ..., n-1.

- A variable made inside a function is **local** - invisible once the function returns. Parameters are local too.
- A **traceback** lists the calls from <module> down to the error. Read it bottom to top.
- Functions are worth it: naming, no repetition, separate testing, reuse.
- Chapter 3 functions only print. Handing a value back (return) is chapter 6.

18. Glossary

Term	Meaning
<code>def</code>	Keyword that starts a function definition
function definition	The <code>def</code> statement that creates a function
header	The first line of a definition (ends in <code>:</code>)
body	The indented lines that run when it is called
function object	The value a <code>def</code> creates; its type is <code>function</code>
call	Running a function: its name followed by <code>()</code>
parameter	A name in the definition standing for an input
argument	A value passed to the function in the call
positional parameter	Matched by where it sits in the call
named parameter	Has a default, so the call may omit it
docstring	A string as the first body line; <code>help</code> shows it
<code>help(f)</code>	Prints a function's signature and docstring
refactoring	Rewriting working code cleaner, same behaviour
for loop	Runs its body once per value in a sequence
<code>range(n)</code>	The sequence <code>0, 1, ..., n-1</code>
iteration	One pass through the body of a loop
dummy variable	A loop variable the body never uses (often <code>_</code>)
local variable	A variable that lives only inside its function
scope	The part of a program where a name is usable
traceback	The chain of calls that were running at an error
stack	The chain of active calls, innermost last

Author: Marcus Birkenkrahe

Created: 2026-09-07 Mon 20:47